

source URL to pull updates. To view the URL that is being used, run `cat linux-2.6/.git/config`.

Working with your local copy

Once you have your local copy, you can create your own branches, modify code, and apply your changes back. Let's explore a few commands to familiarise ourselves with Git usage.

To list all branches in the current repository, issue the command given below:

```
$ git branch
```

The current branch will have a '*' before it. The command `git status` also shows the current branch. To create a new branch, and make it the working branch, use the following command:

```
$ git checkout -b new_branch
```

To jump from the current branch to a new branch, use the code given below:

```
$ git checkout new_branch
```

To change code and commit it into our local repository, the general steps are as follows:

```
$ cd linux-2.6
# enter the repository
do some changes in the files
$ git commit -a
```

When you commit changes, Git will prompt you to enter a commit edit message (in vi). Once you enter one and save it, the changes are committed.

Some more common commands and their purpose:

<code>git log</code>	To view a log of commits. Your most recent commit will be the top-most.
<code>git log -p</code>	Shows the difference in the form of patches.
<code>git add <file-name></code>	This is used to add new files to the repository.
<code>git rm <file></code>	This is to remove files. You should not use the Linux 'rm' command directly from within the repository.
<code>git log master..<current_branch></code>	Shows the difference between the current branch and the master branch. Instead of <code>current_branch</code> you can also use <code>HEAD</code> .
<code>git log master..Head <your-recently-changed-file></code>	To see the differences in a file that has recently been changed.

<code>git diff</code>	Shows all changes; can be redirected to a patch file and sent out. This should be used before a Git commit.
<code>git gc</code>	Optimises the repository; helps boost performance.

If you have done your changes in a different branch and want to merge them into the master branch, you can try the following command:

```
$git checkout master # first enter into the master branch
$git merge new_branch # then issue merge.
```

If you have done a lot of changes, but not performed a commit yet, and want to flush them off the branch, try the command that follows:

```
$git checkout -f
```

If you have already committed, and want to get rid of the commit, use the following code:

```
$ git reset HEAD-1 # last commit is removed.
```

Contributing to Git development

Git's core development is done with Git itself. It is still in active development, and hence commits are actively pushed. The latest commits can be viewed on the Web at [git://git.kernel.org/pub/scm/git/git.git](http://git.kernel.org/pub/scm/git/git.git). You can clone the latest copy with the following command:

```
# git clone git://git.kernel.org/pub/scm/git/git.git
```

If you think you can contribute to Git development, clone your copy, and start working on it!

This article covers the most commonly used set of Git commands. For more information and advanced usage, see the book mentioned in the *References* section. It is not an absolute must that you use Git to do kernel development, but like any other tool, it helps you speed up your work, and allows you to concentrate on the actual patch rather than on how to create a patch. **END** 

References

- [1] The Git book is the best source of information; it's at <http://book.git-scm.com/>.

By: Surya Prabhakar

The author is an engineering advisor in the Product Group at the Dell India R&D Centre, Bengaluru, and has eight years of experience in Linux. He spends most of his time hacking and playing around with Linux. He can be reached at surya_prabhakar@dell.com.